

Omni-Compass against every major cluster platform: A, B and C

Referee report, 26 September 2026. Repository Omni-Compass/The-Omni-Compass-Control-Core-Engine, branch claude/kubernetes-clusters-docker-stack-gp26ve, commit 096956a. Every number is generated by `python tools/abc_report.py` from result files in the repository. Each result states how it was obtained: **measured** on a live Kubernetes control plane, **simulated** in this repository's plants, or **emulated** (a competitor reproduced from its public documentation, not its binary).

Engine interpretation notice. The Omni-Compass engine is the completed core conveyance mechanism; its success criterion is internal dual-basin conveyance under native ignition dynamics. Everything measured here (the Kubernetes adapters, the closure governor, the vendor comparisons) is wiring around the engine, which is downstream engineering and not part of the base law. A loss in a cell below is a property of that wiring on that plant, not evidence that the base engine is incomplete. The engine source is byte-locked by SHA-256 and was not edited for any result in this report.

1. The answer

Three architectures were measured against seven platforms: Kubernetes, Red Hat OpenShift, Google GKE, Azure AKS (node auto-provisioning, which is Karpenter and also stands for Amazon EKS Auto Mode), IBM Turbonomic, CAST AI and Spot Ocean.

- **A. The platform alone.** Its own autoscalers decide.
- **B. Omni-Compass on top of the platform.** The platform's controllers keep running; the six-state engine governs on top.
- **C. Omni-Compass alone.** The engine is the single authority; the platform's managers do not decide.

B against A (held-out, 30 unseen scenarios per workload, settings frozen by SHA-256 first): 363 of 364 gauge-by-platform-by-workload cells are equal or better; 1 is worse: batch jobs on CAST AI, scale reversals 7.7% worse. Batch jobs gain most: the slowest jobs finish much sooner on every platform (section 5.3). Where no intervention was free (web services, four clusters) Omni-Compass reproduces the platform exactly.

C against the seven platforms at once (held-out): 63 cells where some platform is better (section 6). Part of that gap is not closable by any controller: a perfect controller that knows the future cannot have both the tightest packer's machine-hours and the laziest autoscaler's machine churn (section 6.2).

C with the manuscript's closure law and muscle tone (held-out, 30 unseen scenarios per workload, settings frozen first): 41 of 364 cells where some platform is better. Against Kubernetes alone: web services energy +20%, slowest responses (p99) +60%, average response +20%; four clusters, one site energy +22%, slowest responses (p99) -19%, average response +11%; batch jobs energy +6%, slowest responses (p99) +6%, average response -6%; GPU training energy +1%, slowest responses (p99) +0%, average response +0% (positive = better; section 6.4).

Live Kubernetes (measured, run 36221702999, every muscle on, B architecture): worker machines 6.00 to 3.19, energy -3.2% with parked machines on standby, response time p95 491 to 383 ms and p99 613 to 491 ms, 0 failed requests; one significant negative (energy per CPU core-hour, section 7).

2. The rule, and how a cell is judged

- A **cell** is one gauge, one competitor, one workload. There are 13 gauges: energy, machine-hours, response time (p95, p99, mean), time over the backlog, power and heat limits, machine starts+stops, scale reversals, pod changes, work completed and time healthy.
- Omni-Compass **loses** a cell when it is worse by more than 0.5% (0.5 events for counts) **and** the paired bootstrap 95% interval of the per-scenario difference excludes zero. Otherwise the cell is equal (identical, or not significantly different) or better.
- Settings are chosen on **development** scenarios only, frozen with SHA-256 hashes (tuning/B_PREREGISTRATION*.json, results/muscles/PREREGISTRATION.json), then run once on **held-out** scenarios never used for tuning.
- Scenario generator: fleet/harness.py (15-second resolution, 6 hours; boot delay, power model, thermal model, site power limit identical for every arm). Response time: M/M/c queueing per workload plus backlog drain (fleet/sim_slo.py).

3. The opponents, and how each is reproduced

Platform	What decides	Reproduced as (parameters)
Kubernetes	HPA + Cluster Autoscaler	HPA 70%; remove a node after 10 min below 50% request use, not within 10 min of an add
OpenShift	ClusterAutoscaler resource	documented example: threshold 0.4, unneeded 5 min, delay after add 10 min
GKE (optimize-utilization)	MostAllocated packing, aggressive scale-down	declared: threshold 0.65, 2 min (Google publishes no numbers)
AKS NAP (= Karpenter, EKS Auto Mode)	consolidation WhenEmptyOrUnderutilized	consolidateAfter 0 s; remove when the rest hold all requests at 90% packing
IBM Turbonomic	market-based resize + suspend	requests resized every 10 min to per-pod p99 usage; nodes suspended toward 70% packing
CAST AI	Evictor bin-packing	every 60 s, drain a node older than 5 min when its pods fit elsewhere
Spot Ocean	headroom autoscaler	5% automatic headroom kept; least-used node removed when pods fit elsewhere

These are emulations of documented behaviour on the same plant, not the vendors' software; each vendor's full product has more features than its autoscaling core. Every platform runs its pods at the same HPA target (70%), so pod behaviour is identical and differences come from how machines are managed.

4. What Omni-Compass is, mechanically

A six-state dynamical system, $x = (E, U, I_U, S, B, B_{\dot{}})$: energy/excitation, order (a double-well with a healthy basin at $U = +1$), integrated unmet need, stress (a cubic potential), and a damped second-order bath. Equations (1)-(8) and every term's role are in docs/MECHANISM_OF_ACTION.md. Telemetry is blended into the state (weight 0.339) and the equations are integrated with RK4 each decision.

Mode (Jacobian eigenvalue at the operating point)	Time constant (decisions)	Period (decisions)
-7.90+0.00i	1.3	
-3.10+0.00i	3.2	
-0.72+0.00i	13.9	

Mode (Jacobian eigenvalue at the operating point)	Time constant (decisions)	Period (decisions)
-0.20+0.98i	50.0	64
-0.20-0.98i	50.0	64
-0.17+0.00i	59.8	

All modes are stable: the operating point ($U = 0.976$) sits inside the healthy basin. The equations and the observation blend each move the state about half of every step. Compute: 95 microseconds per decision in Python, about 3 in the C++ twin.

5. B: Omni-Compass on top of each platform, against the same platform alone (held-out)

Source `tuning/B_LEAGUE_HELDOUT2.json`. Omni-Compass keeps the platform running and intervenes only where the engine sees it will pay: it undoes a machine removal that is about to be reversed (while requests are rising or the engine has not converged), and adds early the machine the platform would add only after pods go pending. With both off it reproduces the platform exactly (checked on every platform and workload); every gain below is on top of that floor.

5.1 Web services

Setting: pass-through. Every intervention tried here cost some gauge on some platform, so Omni-Compass leaves these decisions to the platform and the result is identical to the platform alone.

Platform	Better (significant)	Worse beyond tolerance (LOSS)	Worse within 0.5% tolerance	Equal
Kubernetes	-	-	-	13 of 13
OpenShift	-	-	-	13 of 13
GKE (optimize)	-	-	-	13 of 13
AKS NAP (Karpenter)	-	-	-	13 of 13
Turbonomic	-	-	-	13 of 13
CAST AI	-	-	-	13 of 13
Spot Ocean	-	-	-	13 of 13

5.2 Four clusters, one site

Setting: pass-through. Every intervention tried here cost some gauge on some platform, so Omni-Compass leaves these decisions to the platform and the result is identical to the platform alone.

Platform	Better (significant)	Worse beyond tolerance (LOSS)	Worse within 0.5% tolerance	Equal
Kubernetes	-	-	-	13 of 13
OpenShift	-	-	-	13 of 13
GKE (optimize)	-	-	-	13 of 13

Platform	Better (significant)	Worse beyond tolerance (LOSS)	Worse within 0.5% tolerance	Equal
AKS NAP (Karpenter)	-	-	-	13 of 13
Turbonomic	-	-	-	13 of 13
CAST AI	-	-	-	13 of 13
Spot Ocean	-	-	-	13 of 13

5.3 Batch jobs

Setting per platform (chosen on development scenarios): Kubernetes: early add (lead 12, trend over 16), reversal veto; OpenShift: early add (lead 12, trend over 16), reversal veto; GKE (optimize): early add (lead 6, trend over 8), reversal veto; AKS NAP (Karpenter): early add (lead 3, trend over 16), reversal veto; Turbonomic: early add (lead 6, trend over 4), reversal veto; CAST AI: early add (lead 12, trend over 16), reversal veto; Spot Ocean: early add (lead 3, trend over 16), reversal veto.

Platform	Better (significant)	Worse beyond tolerance (LOSS)	Worse within 0.5% tolerance	Equal
Kubernetes	response p99 -91%; response mean -23%; machine starts+stops -16%; scale reversals -70%	-	-	9 of 13
OpenShift	response p99 -91%; response mean -25%; machine starts+stops -23%; scale reversals -87%	-	-	9 of 13
GKE (optimize)	response p99 -91%; response mean -27%; machine starts+stops -22%	-	-	10 of 13
AKS NAP (Karpenter)	response p95 -57%; response p99 -24%; response mean -19%; time over backlog limit -47%; machine starts+stops -13%	-	energy +0.2%; machine-hours +0.5%	6 of 13
Turbonomic	response p99 -95%; response mean -34%; machine starts+stops -16%	-	-	10 of 13
CAST AI	response p95 -47%; response p99 -25%; response mean -32%; time over backlog limit -100%; machine starts+stops -9%	scale reversals +7.7%	-	7 of 13
Spot Ocean	response p95 -12%; response p99 -9%; response mean -8%; time over backlog limit -100%; machine starts+stops -8%; scale reversals -8%	-	energy +0.1%; machine-hours +0.3%	5 of 13

5.4 Gpu training

Setting per platform (chosen on development scenarios): Kubernetes: early add (lead 3, trend over 8), reversal veto; OpenShift: early add (lead 3, trend over 16); GKE (optimize): pass-through; AKS NAP (Karpenter): pass-through; Turbonomic: early add (lead 3, trend over 8); CAST AI: pass-through; Spot Ocean: pass-through.

Platform	Better (significant)	Worse beyond tolerance (LOSS)	Worse within 0.5% tolerance	Equal
Kubernetes	-	-	-	13 of 13
OpenShift	-	-	time over heat limit +0.1%	12 of 13
GKE (optimize)	-	-	-	13 of 13
AKS NAP (Karpenter)	-	-	-	13 of 13

Platform	Better (significant)	Worse beyond tolerance (LOSS)	Worse within 0.5% tolerance	Equal
Turbonomic	-	-	energy +0.1%; time over heat limit +0.2%; machine-hours +0.4%	10 of 13
CAST AI	-	-	-	13 of 13
Spot Ocean	-	-	-	13 of 13

Percentages are the plain change of Omni-Compass on top against the platform alone: for response times, starts+stops, reversals, energy and machine-hours negative is better; for work completed and time healthy positive is better. A LOSS is worse beyond the 0.5% tolerance with a 95% interval excluding zero (section 2).

6. C: Omni-Compass alone, against all seven platforms at once

Source tuning/LEAGUE_HELDOUT.json. One Omni-Compass configuration per workload type (chosen on development scenarios): web: speed159, multi: speed159, batch: speed112, gpu: omni_fleet.

Workload	Gauge	Platforms better than Omni-Compass	Omni worse by (range)
batch	energy	GKE (optimize), AKS NAP (Karpenter), Turbonomic, CAST AI, Spot Ocean	1.7% to 6.7%
batch	machine-hours	GKE (optimize), AKS NAP (Karpenter), Turbonomic, CAST AI, Spot Ocean	4.4% to 18.0%
batch	scale reversals	Kubernetes, OpenShift	266.7% to 279.3%
batch	machine starts+stops	Kubernetes, OpenShift	54.3% to 64.9%
gpu	machine-hours	GKE (optimize), AKS NAP (Karpenter), Turbonomic, CAST AI, Spot Ocean	5.5% to 14.4%
gpu	scale reversals	Kubernetes, OpenShift	85.6% to 87.6%
gpu	machine starts+stops	Kubernetes, OpenShift	37.6% to 49.8%
multi	energy	Kubernetes, OpenShift, GKE (optimize), AKS NAP (Karpenter), Turbonomic, CAST AI, Spot Ocean	23.9% to 64.7%
multi	machine-hours	Kubernetes, OpenShift, GKE (optimize), AKS NAP (Karpenter), Turbonomic, CAST AI, Spot Ocean	36.7% to 119.0%
multi	pod changes	Kubernetes, OpenShift, GKE (optimize), AKS NAP (Karpenter), CAST AI, Spot Ocean	8.2% to 10.3%
web	energy	Kubernetes, OpenShift, GKE (optimize), AKS NAP (Karpenter), Turbonomic, CAST AI, Spot Ocean	23.9% to 63.5%
web	machine-hours	Kubernetes, OpenShift, GKE (optimize), AKS NAP (Karpenter), Turbonomic, CAST AI, Spot Ocean	36.8% to 116.3%
web	pod changes	Kubernetes, OpenShift, GKE (optimize), AKS NAP (Karpenter), CAST AI, Spot Ocean	7.8% to 10.8%

6.1 Where C wins

- Web services: response p95 110.1 ms vs best platform 124.9 ms; response p99 122.8 ms vs best platform 546 ms; response mean 145.3 ms vs best platform 216.2 ms.
- Four clusters, one site: response p95 112.1 ms vs best platform 126.9 ms; response p99 137.8 ms vs best platform 262.9 ms; response mean 145.6 ms vs best platform 217.2 ms.
- Batch jobs: response p99 337.4 ms vs best platform 1277 ms; response mean 115.2 ms vs best platform 135.6 ms.

6.2 The physical limit: what no controller can do

For each scenario, dynamic programming with perfect knowledge of the future (boot delay ignored, which only helps it) finds the fewest machine starts+stops that can hold every pod at a given machine-hour budget (tuning/bound.py).

Workload	Scenario	Tightest platform machine-hours	Fewest starts+stops of any platform	Perfect controller at those machine-hours
web	101	18.4	9	11
web	102	18.3	9	9
web	103	31.4	4	26
web	104	22.0	12	24
multi	101	74.9	36	46
multi	102	112.7	26	100
multi	103	122.5	17	114
multi	104	93.4	35	91
batch	101	108.0	24	67
batch	102	56.8	7	33
batch	103	66.9	12	44
batch	104	93.4	15	58
gpu	101	66.4	12	28
gpu	102	57.5	16	36
gpu	103	51.5	19	30
gpu	104	57.0	20	26

In 15 of 16 scenarios even a perfect controller needs more starts+stops than the laziest platform to reach the tightest platform's machine-hours. Those two platforms sit at opposite ends of one physical trade; beating both at once is impossible for any software. In C, a loss on one side of that pair is the price of a win on the other.

6.3 Letting the equations drive directly

A governor in which the equations run closed-loop and their own quantities (control effort, unmet need I_U , the bath's rate) drive every muscle (omnicompass/mathdrive.py) was tested on 96 settings per workload: 66 losing cells, the same trade-off as the rule-based wiring. The engine clock alone moved batch p99 from 3.9 s to 0.1 s at equal energy.

6.4 Strict C: the manuscript's closure law drives the machines and the pods

omnicompass/closure.py implements, as the only authority over machines, the laws of the owner's manuscript: the forward projection of Section 5c, the master closure law of Chapter 20 ($F = G_0 + G_c$: no correction inside the admissible domain, an inward correction sized to restore the margin when the projected state approaches its boundary, so that $G \cdot n \leq 0$ on the boundary), the turning point of Chapters 29-30 (a machine is released only after the peak) and the dual-bath exchange of Chapter 31 (level and rate of demand). A release must also stay inside the calm set for a dwell (the resource-aware envelope, Proposition 2). Kubernetes' HPA and node managers are

off; Kubernetes only schedules. Settings chosen on development scenarios, frozen by SHA-256 (56be5f50b8d2631c), then run once on 30 held-out scenarios per workload.

Workload	Gauge	Kubernetes	OpenShift	GKE (optimize)	AKS NAP (Karpenter)	Turbonomic	CAST AI	Spot Ocean	Omni-Compass (closure law)
web	energy	14.11	14.75	11.8	11.22	11.75	11.03	11.2	11.26
web	machine-hours	37.61	40.27	27.99	25.56	27.78	24.77	25.52	24.26
web	response p95	122.9	122.8	123.4	123.6	138.2	124.1	123.8	140.4
web	response p99	806.6	648.3	1345	1428	3251	2500	2023	318.9
web	response mean	206.4	205.3	212.5	215.7	251	236.1	226.9	164.5
web	machine starts+stops	8.733	6.5	16.47	19.67	12.73	26.57	27.6	11.6
web	work completed	1	1	1	1	1	1	1	1
multi	energy	54.24	56.4	45.04	42.81	44.86	42.16	42.85	42.43
multi	machine-hours	145.7	154.7	107.4	98.08	106.6	95.37	98.24	90.42
multi	response p95	124.9	124.8	125.6	125.9	119	126.5	126.1	148.9
multi	response p99	202.2	168.7	641.9	991.4	2.095e+04	2068	1531	240.3
multi	response mean	220.2	218.4	236.9	243.1	784.9	261.8	253.7	196.8
multi	machine starts+stops	36.77	28	66.63	79.7	53.8	102.4	108.1	45.67
multi	work completed	1	1	1	1	1	1	1	1
batch	energy	65.45	65.68	63.28	60.11	62.73	60.65	60	61.65
batch	machine-hours	94.71	95.49	87.48	76.91	85.67	78.71	76.55	78.5
batch	response p95	100	100	100	861.9	100	1840	220.1	100
batch	response p99	1072	1291	2086	3880	2242	4974	2491	1012
batch	response mean	133.1	138	158.8	257.2	170.6	360	184.2	141.3
batch	machine starts+stops	12.93	14.13	29	92.33	35.03	160.6	141.2	17.2
batch	work completed	1	1	1	1	1	1	1	1
gpu	energy	201.4	201.6	199.5	197.2	198.9	197.3	197	198.9
gpu	machine-hours	60.77	61.07	58.15	54.89	57.25	55.06	54.62	55.29
gpu	response p95	2.288e+06	2.286e+06	2.288e+06	2.29e+06	2.288e+06	2.292e+06	2.29e+06	2.287e+06
gpu	response p99	2.397e+06	2.396e+06	2.398e+06	2.4e+06	2.398e+06	2.402e+06	2.401e+06	2.396e+06
gpu	response mean	8.886e+05	8.875e+05	8.894e+05	8.904e+05	8.886e+05	8.922e+05	8.911e+05	8.879e+05
gpu	machine starts+stops	16.5	16.6	33.5	50.63	30.57	53.93	55.63	15.27
gpu	work completed	0.9027	0.9028	0.9027	0.9026	0.9028	0.9025	0.9026	0.9028

Losing cells (worse than that platform beyond tolerance): web services 10 of 91; four clusters, one site 12 of 91; batch jobs 7 of 91; GPU training 12 of 91.

- Web services: response p95 vs Kubernetes 14.2% worse; machine starts+stops vs Kubernetes 32.8% worse; response p95 vs OpenShift 14.3% worse; machine starts+stops vs OpenShift 78.5% worse; scale reversals vs OpenShift 192.9% worse; response p95 vs GKE (optimize) 13.8% worse; response p95 vs AKS NAP (Karpenter) 13.6% worse; energy vs CAST AI 2.1% worse; response p95 vs CAST AI 13.2% worse; response p95 vs Spot Ocean 13.4% worse.

- Four clusters, one site: response p95 vs Kubernetes 19.2% worse; machine starts+stops vs Kubernetes 24.2% worse; response p95 vs OpenShift 19.4% worse; response p99 vs OpenShift 42.4% worse; machine starts+stops vs OpenShift 63.1% worse; scale reversals vs OpenShift 153.2% worse; response p95 vs GKE (optimize) 18.5% worse; response p95 vs AKS NAP (Karpenter) 18.3% worse; response p95 vs Turbonomic 25.2% worse; energy vs CAST AI 0.6% worse; response p95 vs CAST AI 17.8% worse; response p95 vs Spot Ocean 18.1% worse.
- Batch jobs: machine starts+stops vs Kubernetes 33.0% worse; machine starts+stops vs OpenShift 21.7% worse; energy vs AKS NAP (Karpenter) 2.6% worse; machine-hours vs AKS NAP (Karpenter) 2.1% worse; energy vs CAST AI 1.7% worse; energy vs Spot Ocean 2.8% worse; machine-hours vs Spot Ocean 2.6% worse.
- Gpu training: energy vs AKS NAP (Karpenter) 0.9% worse; time over power limit vs AKS NAP (Karpenter) 0.7% worse; time over heat limit vs AKS NAP (Karpenter) 0.8% worse; machine-hours vs AKS NAP (Karpenter) 0.7% worse; energy vs CAST AI 0.8% worse; time over power limit vs CAST AI 1.5% worse; time over heat limit vs CAST AI 1.9% worse; energy vs Spot Ocean 1.0% worse; time over power limit vs Spot Ocean 1.8% worse; time over heat limit vs Spot Ocean 1.6% worse; machine-hours vs Spot Ocean 1.2% worse; time healthy vs Spot Ocean 1.5% worse.

6A. Runtime Benchmark Protocol: faults at five stress levels

The owner's *OmniCompass Runtime Benchmark Protocol* (stage 1, Python runtime) run on this fleet plant with every architecture: 100 runs per stress level per workload, seeds 900001-900100 (never used for tuning). Level L injects L faults (machines dying, load spikes, services crash-looping, a noisy neighbour), identical for every system (tools/protocol_bench.py). A run is **conveyed** when every fault is recovered within 20 minutes and every cluster is inside the admissible basin for the final 10 minutes (worst response <= 1 s, queue < 0.28, pending pods <= 5%, power and heat inside limits). A **page** is an out-of-basin episode of 5 minutes or more, the usual alert rule: each would call a human. Nobody intervenes in the simulation.

Web services

Column	System	Conveyed	Pages	Recovery (min)	Minutes outside basin	Timeouts > 2 s	Energy (kWh)
A	AKS NAP (Karpenter)	233 of 500	391	6.6	32.8	4.52%	11.90
B	AKS NAP (Karpenter)	233 of 500	391	6.6	32.8	4.52%	11.90
A	CAST AI	231 of 500	404	6.6	34.1	4.68%	11.75
B	CAST AI	231 of 500	404	6.6	34.1	4.68%	11.75
A	GKE (optimize)	227 of 500	366	6.9	32.4	4.25%	12.76
B	GKE (optimize)	227 of 500	366	6.9	32.4	4.25%	12.76
A	Kubernetes	218 of 500	338	8.3	33.3	3.97%	15.54
B	Kubernetes	218 of 500	338	8.3	33.3	3.97%	15.54
A	OpenShift	220 of 500	328	8.0	32.2	3.82%	15.52
B	OpenShift	220 of 500	328	8.0	32.2	3.82%	15.52
A	Spot Ocean	234 of 500	392	6.6	32.8	4.56%	11.94
B	Spot Ocean	234 of 500	392	6.6	32.8	4.56%	11.94
A	Turbonomic	169 of 500	552	14.4	53.3	5.96%	12.43
B	Turbonomic	169 of 500	552	14.4	53.3	5.96%	12.43
C	C-hpa	202 of 500	353	11.7	35.8	3.81%	20.69
C	C-strict	238 of 500	283	6.0	26.2	3.45%	13.02

In the protocol's required form: Omni-Compass alone (closure law) conveyed 238 out of 500 runs; the best platform alone (Spot Ocean) conveyed 234 out of 500. Omni-Compass would have paged a human 283 times, that platform 392 times. Omni-Compass average recovery time was 6.0 min, that platform's 6.6 min.

Four clusters, one site

Column	System	Conveyed	Pages	Recovery (min)	Minutes outside basin	Timeouts > 2 s	Energy (kWh)
A	AKS NAP (Karpenter)	49 of 500	534	5.6	69.5	1.71%	45.25
B	AKS NAP (Karpenter)	49 of 500	534	5.6	69.5	1.71%	45.25
A	CAST AI	44 of 500	608	5.7	74.2	1.81%	44.54
B	CAST AI	44 of 500	608	5.7	74.2	1.81%	44.54
A	GKE (optimize)	49 of 500	455	5.7	65.6	1.63%	47.96
B	GKE (optimize)	49 of 500	455	5.7	65.6	1.63%	47.96
A	Kubernetes	44 of 500	330	6.5	59.2	1.44%	57.67
B	Kubernetes	44 of 500	330	6.5	59.2	1.44%	57.67
A	OpenShift	43 of 500	310	6.4	57.6	1.38%	59.47
B	OpenShift	43 of 500	310	6.4	57.6	1.38%	59.47
A	Spot Ocean	44 of 500	508	5.6	69.1	1.73%	45.30
B	Spot Ocean	44 of 500	508	5.6	69.1	1.73%	45.30
A	Turbonomic	26 of 500	867	15.0	97.6	2.49%	47.57
B	Turbonomic	26 of 500	867	15.0	97.6	2.49%	47.57
C	C-hpa	64 of 500	328	8.8	45.8	1.07%	74.42
C	C-strict	45 of 500	231	4.3	52.6	1.33%	45.82

In the protocol's required form: Omni-Compass alone (closure law) conveyed 45 out of 500 runs; the best platform alone (GKE (optimize)) conveyed 49 out of 500. Omni-Compass would have paged a human 231 times, that platform 455 times. Omni-Compass average recovery time was 4.3 min, that platform's 5.7 min.

Batch jobs

Column	System	Conveyed	Pages	Recovery (min)	Minutes outside basin	Timeouts > 2 s	Energy (kWh)
A	AKS NAP (Karpenter)	314 of 500	234	4.5	31.4	9.01%	70.02
B	AKS NAP (Karpenter)	338 of 500	235	4.5	25.1	8.09%	70.95
A	CAST AI	161 of 500	237	4.4	39.9	10.25%	72.15
B	CAST AI	224 of 500	233	4.4	26.3	8.39%	73.91
A	GKE (optimize)	409 of 500	235	4.0	19.0	7.07%	77.00
B	GKE (optimize)	425 of 500	228	4.0	13.9	6.38%	79.12
A	Kubernetes	424 of 500	228	3.7	15.4	6.28%	85.35
B	Kubernetes	425 of 500	234	3.8	12.2	5.77%	87.43
A	OpenShift	422 of 500	232	3.8	16.0	6.36%	85.36
B	OpenShift	425 of 500	232	3.8	12.2	5.82%	86.89
A	Spot Ocean	360 of 500	231	4.4	23.1	7.77%	71.08

Column	System	Conveyed	Pages	Recovery (min)	Minutes outside basin	Timeouts > 2 s	Energy (kWh)
B	Spot Ocean	360 of 500	234	4.4	20.6	7.56%	71.79
A	Turbonomic	407 of 500	232	4.2	21.0	7.45%	74.08
B	Turbonomic	426 of 500	233	4.1	14.2	6.62%	75.25
C	C-hpa	166 of 500	235	30.8	73.5	20.65%	85.38
C	C-strict	422 of 500	237	4.1	17.3	6.93%	74.13

In the protocol's required form: Omni-Compass alone (closure law) conveyed 422 out of 500 runs; the best platform alone (Kubernetes) conveyed 424 out of 500. Omni-Compass would have paged a human 237 times, that platform 228 times. Omni-Compass average recovery time was 4.1 min, that platform's 3.7 min.

Gpu training

Column	System	Conveyed	Pages	Recovery (min)	Minutes outside basin	Timeouts > 2 s	Energy (kWh)
A	AKS NAP (Karpenter)	13 of 500	802	95.4	227.2	69.96%	202.42
B	AKS NAP (Karpenter)	13 of 500	802	95.4	227.2	69.96%	202.42
A	CAST AI	13 of 500	925	95.1	226.5	70.61%	202.80
B	CAST AI	13 of 500	925	95.1	226.5	70.61%	202.80
A	GKE (optimize)	15 of 500	802	96.5	227.1	68.76%	205.17
B	GKE (optimize)	15 of 500	802	96.5	227.1	68.76%	205.17
A	Kubernetes	18 of 500	782	97.5	225.4	67.95%	206.83
B	Kubernetes	18 of 500	792	97.6	226.1	67.87%	207.04
A	OpenShift	18 of 500	788	97.6	226.3	67.91%	207.08
B	OpenShift	18 of 500	788	97.6	226.2	67.88%	207.13
A	Spot Ocean	13 of 500	830	94.9	226.0	70.66%	202.22
B	Spot Ocean	13 of 500	830	94.9	226.0	70.66%	202.22
A	Turbonomic	14 of 500	786	96.5	226.6	68.78%	204.30
B	Turbonomic	14 of 500	792	96.2	226.6	68.67%	204.40
C	C-hpa	0 of 500	1232	174.0	295.6	83.41%	198.48
C	C-strict	13 of 500	809	95.1	225.0	67.86%	204.08

In the protocol's required form: Omni-Compass alone (closure law) conveyed 13 out of 500 runs; the best platform alone (Kubernetes) conveyed 18 out of 500. Omni-Compass would have paged a human 809 times, that platform 782 times. Omni-Compass average recovery time was 95.1 min, that platform's 97.5 min.

6B. Second generation (held-out, settings frozen by SHA-256 before each run)

B with muscle tone and consolidation (tuning/b_tone.py, held-out seeds 700401-700430): machines a platform powers off are parked instead (alive, low power, instant wake), with a reserve sized by the law; Omni consolidates on top only where that loses nothing. Losing cells: 3 of 364.

B fix for the remaining pairs (tuning/b_fix.py, 30 development seeds, held-out 700701-700730): multi on aks_nap: 0 losing cells; batch on turbonomic: 0 losing cells; multi on cast_ai: 0 losing cells.

Whole body at four-cluster sites (tuning/site_league.py, held-out 701001-701030): traffic shift between a site's clusters (a cluster short of room runs its overflow on another cluster's powered spare cores, +5 ms per shifted request; assumes replicated services) and, for C, the closure law on the site total with one warm reserve.

Platform	B losing cells	C losing cells	C energy	C slowest responses	C average
Kubernetes	0	2	+16%	+56%	+30%
OpenShift	0	2	+19%	+46%	+29%
GKE (optimize)	0	0	+7%	+85%	+36%
AKS NAP (Karpenter)	0	0	+2%	+90%	+39%
Turbonomic	0	1	+7%	+99%	+75%
CAST AI	0	0	-0%	+94%	+42%
Spot Ocean	0	0	+2%	+91%	+40%

C gains are Omni against that platform (positive = Omni better).

6C. Live levers on real Kubernetes (measured)

Live levers on real Kubernetes (kind, 1 control plane + 6 workers), GitHub Actions run 36270589830, commit 43f6747, 2026-09-26 20:45-20:48 UTC. Omni-Compass ran as its least-privilege service account (deploy/kind/rbac-omni.yaml + rbac-levers.yaml). Script: scripts/kind_levers.sh. Result: LIVE LEVERS: PASS, 19 of 19 checks, 0 lever errors.

Permission receipts (kubectl auth can-i as the Omni service account)

```
can:      patch pods/resize default, patch deployment/idle-worker, patch deployments/scale idle-worker,
          get configmap/omni-work, patch jobs default, create/delete resourcequotas agents, patch pods/resize agents
cannot:   delete pods agents, create pods agents, delete deployments agents, patch deployment/load-generator,
          create resourcequotas default, get secrets (any namespace)
```

```
rightsize  php-apache CPU requests: before [200m] after [50m]; after kill [200m]
coldstart  idle-worker replicas: idle -> 0; work waiting -> 2; idle again -> 0; after kill -> 2
batch_pace train-a suspended: stress -> true; calm -> false; stress -> true; after kill -> false
contain    agent quota present: over budget -> 1; under budget -> 0; over -> 1; after kill -> 0
           agent CPU limits: before [400m 400m] contained [100m 100m] lifted [400m 400m] after kill [400m 400m]
cooling    setpoints written to the stand-in building controller: 27.0 27.0 27.0 22.0 (kill restores 22.0)
records left after kill: none
```

Earlier runs of the same script (kept for the record): run 36268772674 failed (containment could not lift: Kubernetes refuses to raise a pod limit above a standing quota; one failing lever stopped the kill switch); run 36269885518 printed PASS but its checks were chained so failures were not counted (containment still not lifted); run 36270202629 failed in the script's quota probe once the quota was correctly deleted. Fixes: quota deleted before limits are restored; each lever and each kill-switch restore isolated; every check counted on its own line. Not covered: the cooling lever writes to a stand-in controller (a CI runner has no chiller); CPU frequency and power caps need hardware with cpufreq; traffic shift needs two real clusters behind one load balancer.

7. Live Kubernetes (measured)

Two identical kind clusters (1 control plane + 6 workers) at the same time, same load; one with Omni-Compass on top (B). Omni runs as a least-privilege service account whose permissions are proven with `kubectl auth can-i` receipts before each run.

Run	Change	Validity	p95 (ms)	p99 (ms)	Energy
36213152881	HPA target, node pool, power cap (in-place pod CPU limits), heat (harn	valid	486 to 802	675 to 1,113	222 to 215
36214629046	latency afferent (p95 over 500 ms SLO as queue pressure) and power-cap	valid	481 to 600	596 to 777	222 to 219
36216647786	SLO reflex; eviction receipt fixed	INVALID as a test of the engine	479 to 320	583 to 420	224 to 220
36218637030	controller fail-safe; benchmark prints controller log and rejects earl	PARTIAL	491 to 492	645 to 683	223 to 221
36220059046	get on pods/resize (power cap can apply)	INVALID for response time and energy	501 to 380	660 to 415	224 to 220
36221702999	probe tunnel restarts on both arms; power cap able to apply	VALID	491 to 383	613 to 491	224 to 216

Run 6 in full:

Gauge	Kubernetes alone	+ Omni-Compass	Change
Worker nodes in service, mean	6.00	3.19	-46.8%
Node-hours	2.02	1.07	-47.0%
Energy (Wh), standby counted	224	216	-3.2%
Power (W), peak	710	682	-3.8%
Energy per core-hour (Wh)	742	1,041	+40.2% worse (significant)
Utilisation	0.074	0.097	+30.2%
Pending pods, pod-minutes	0.283	0.533	+88% (n.s.)
HPA shortfall, minutes	0.817	1.33	+63% (n.s.)
Response time (ms), mean	264	165	-37.4%
Response time (ms), median	227	119	-47.4%
Response time (ms), 95th percentile	491	383	-22.1%
Response time (ms), 99th percentile	613	491	-19.9%
Failed requests (%)	0	0	0

First fully valid run with every live muscle on: fewer servers (-47%), less energy (-3.2%, standby counted), lower peak power, and faster responses at every percentile. The one significant negative is energy per core-hour (+48%): the power cap lowers the CPU the app consumes (-31%), and that CPU is the metric's denominator. Whether energy per request served also rose is not measured (the load generator's request count is not captured); a per-request energy gauge is added to the capture next.

8. The nine problem-map muscles (simulated, held-out, Omni-Compass direct against the strongest native tool)

Muscle	Opponent	Better	Worse
rightsize	hpa_vpa	cpu core hours -3%; mem gib hours -11%; p99 ms -99%; replica reversals -84%; slo breach min -57%	p95 ms +15%; oom kills +531%
coldstart	keda	p95 ms -75%; p99 ms -58%; delayed 1s pct -71%; instance hours -8%	cold starts +736%
gpupack	binpack	energy kwh -2%; idle gpu hours powered -20%	wait mean min +16%; migrations 0 to 6.07
powersmooth	floor_safe	energy overhead pct -9%; throughput loss pct -44%; swing 1s mw -6%	max ramp mw s +127%
health	detect	goodput pct +10%; lost gpu hours -20%; restarts -29%; straggler node hours -99%	-
cooling	reset	pue -2%; cooling mwh -13%; inlet violation min -73%	-
inference	keda	ttft p95 s -18%; slo breach pct -58%; gpu hours -19%; preemptions -56%	-
containment	static	rogue overspend usd -96%; time to contain min -99%; peak subagents -59%	false stops +357%; honest work pct -7%
vmenergy	ratio	attr error pct -37%; worst vm error pct -34%	-

9. Threats to validity

- Competitors are emulated from documentation on a shared plant; their production binaries may behave differently, and their full products include features (spot pricing, instance-type selection, rebalancing) that this plant does not model.
- Simulated plants are fluid models: pods pack perfectly, boot delay is fixed, power is idle + dynamic x utilisation. Real clusters fragment and boot times vary.
- Live runs use kind: nodes are containers on one CI machine; power is modelled, not metered; the two arms ran on two different CI machines. Six 20-minute runs are evidence of behaviour, not of production savings.
- Development and held-out scenarios come from the same generator; held-out protects against tuning to seeds, not against a generator that differs from real traffic. Real traces (PlanetLab) are covered in the main benchmark report.
- The loss rule uses a 0.5% tolerance and a 95% interval on 30 scenarios; small true differences can be missed.

10. Reproduce

```

pip install -r requirements.txt && python verify.py
python tuning/b_league.py dev # B vs A, choose settings (development seeds)
python tuning/b_league.py heldout tuning/B_SETTINGS_FROZEN.json # B vs A, held-out
python tuning/league.py heldout # C vs all platforms
python tuning/bound.py # perfect-foresight frontier
python tools/mechanism.py # engine modes and compute
python tuning/closure_search.py && python tuning/closure_search2.py && python tuning/closure_heldout.py # strict C, closure law
python tools/protocol_bench.py 100 # runtime protocol, faults at five stress levels
for m in rightsize coldstart gpupack powersmooth health cooling inference containment vmenergy; do python -m omnilab.bench $m heldout; done
GitHub Actions workflow 'benchmark' (commit message tag [bench]): live A vs B on kind
python tools/abc_report.py && python pilot/bench_pdf.py docs/OMNICOMPASS_ABC_REPORT.md docs/OMNICOMPASS_ABC_REPORT.pdf

```